Notes: Kafka Architecture - Part 2

Event Driven Architecture

1 Consumers and Consumer Groups

- **Consumer Groups:** Consumers are logically grouped together. A consumer can join any group (e.g., a "Notification" group or an "Analytics" group).
- **Rule 1 (Within a Group):** Different consumers in the *same* group can only read from *different* partitions. Multiple consumers in the same group cannot read the same partition simultaneously.
- **Rule 2 (Across Groups):** Consumers in *different* groups can read from the *same* partition independently.

2 Offset Management (`__consumer_offsets`)

- Each consumer group maintains its partition-wise offsets independently. It tracks till which offset events have been successfully processed.
- **Group-Wise Storage:** Offsets are stored per consumer group, not per individual consumer. This ensures that if a consumer crashes, a new one can take over and know exactly where to resume from by checking the group's committed offset.
- **Where are offsets stored?** Kafka internally creates a special topic called `__consumer_offsets` (by default, it has 50 partitions). All offset data is stored here just like regular topic messages.
- **Partition Assignment for Offsets:** To determine which partition of the `__consumer_offsets` topic holds a specific group's offset, Kafka hashes the Group ID and performs a modulo operation against the number of partitions (e.g., Hash(GroupID) % 50).

3 Offset Commit Strategies

- **Auto-Commit:** Kafka automatically commits the current offset at regular time intervals (e.g., every 5 seconds).
  - *Risk:* If a consumer processes events but crashes before the auto-commit interval, or if auto-commit happens before full processing finishes, data loss or duplicate processing can occur.
- **Manual Commit (Recommended):** The application explicitly tells Kafka when to commit. Typically done after fully processing a batch of polled events.

4 Kafka Cluster & High Availability

- **Kafka Cluster:** A group of Kafka brokers working together to provide scalability, fault tolerance, and high availability. There is no single point of failure.
- **Replication Factor:** Defines how many copies of a partition should exist across the cluster. For example, a replication factor of 2 means 1 Leader and 1 Follower.

5 Leader and Follower Partitions

Partitions are distributed across multiple brokers. Each partition has one Leader and zero or more Followers.

- **Leader Responsibilities:** The Leader handles *all* read and write requests from producers and consumers. It maintains the partition logs and coordinates with followers.
- **Follower Responsibilities:** Followers act as backups. Their only job is to replicate data and stay in sync with the Leader. Followers *never* serve clients. If a Leader fails, a Follower takes over as the new Leader.
- **Broker Distribution:** A single broker can host the Leader for one partition (e.g., Partition 0) while simultaneously hosting the Follower for a completely different partition (e.g., Partition 1).